

EmergencyAI - Smart Emergency Response System

Vibe-a-Thon 2026 Hackathon Project

An AI-powered web application that enables the public to report emergencies quickly, accurately, and efficiently. The system uses advanced AI to analyze emergency reports, classify them into appropriate categories, determine severity levels, and dispatch the right response teams instantly.



Figure 1: EmergencyAI Banner

Live Demo

Frontend: <https://6y7vkgqpf6w4u.ok.kimi.link>

Features

Core Features

- **AI-Powered Emergency Classification:** Automatically analyzes text descriptions to classify emergencies into Police, Ambulance, and/or Fire categories
- **Severity Detection:** Determines severity levels (Low, Medium, High, Critical) based on incident details
- **Automated Alert Generation:** Creates structured, timestamped messages for emergency teams
- **Real-Time Status Tracking:** Track emergency response progress from request to resolution
- **Multi-Media Support:** Upload images and videos as supporting evidence

Advanced Features

- **Voice Input Support:** Report emergencies hands-free using voice commands
- **Live Location Sharing:** Automatic GPS coordinates capture for precise location tracking
- **AI Chatbot Assistant:** 24/7 support for first aid guidance
- **Responder Dashboard:** Full incident management for emergency responders
- **Records Viewer:** View all submitted emergency reports with details
- **Demo Mode:** Works without backend - all features functional!

Status Workflow

1. Request Received
2. Preparing
3. Team Dispatched
4. On the Way
5. Action in Progress
6. Resolved

How AI Classification Works

Step 1: Keyword Analysis

The system scans your emergency description for keywords:

Service	Keywords
 Police	crime, robbery, assault, accident, theft, violence, gun, weapon, threat
 Ambulance	injury, bleeding, heart attack, stroke, unconscious, seizure, pain
 Fire	fire, smoke, gas leak, explosion, trapped, burning, chemical spill

Step 2: Severity Assessment

Based on detected keywords, severity is determined:

Level	Indicators	Response Time
 Critical	shooting, cardiac arrest, severe bleeding, unconscious	< 5 min
 High	assault, serious injury, poisoning, seizure	< 10 min
 Medium	theft, minor injury, fight, asthma attack	< 20 min
 Low	suspicious activity, trespassing, traffic violation	< 30 min

Step 3: Instant Dispatch

The system automatically:

-  Classifies emergency type (Police/Ambulance/Fire)
-  Assigns priority level
-  Generates structured alert message
-  Creates trackable record
-  Shows real-time status updates

Example

Input: “There’s a car accident on Main Street. Someone is bleeding heavily and unconscious.”

AI Output:

- 🚓 Police: (car accident)
- 🚑 Ambulance: (bleeding, unconscious)
- 🚒 Fire:
- **Severity:** Critical
- **Confidence:** 92%

Tech Stack

Frontend

- **React 18** with TypeScript
- **Tailwind CSS** for styling
- **shadcn/ui** components
- **Lucide React** for icons

Backend (Optional - Demo Mode Works Without It!)

- **Python Flask** REST API
- **Flask-CORS** for cross-origin support

Demo Mode

The frontend includes a built-in demo mode that:

- Simulates all API responses
- Stores emergencies in memory
- Provides full functionality without backend
- Perfect for hackathon demos!

Project Structure

```
/mnt/okcomputer/output/  
app/ # React Frontend (Deployed)  
  src/  
    sections/  
      Hero.tsx # Landing hero section  
      EmergencyForm.tsx # Report form with AI analysis  
      AIAnalysis.tsx # Results display  
      StatusTracker.tsx # Live status tracking  
      ResponderDashboard.tsx # Admin dashboard  
      RecordsViewer.tsx # View all records  
      Chatbot.tsx # AI assistant
```

```
    Features.tsx # Features + How it works
    Footer.tsx
    services/api.ts # API with DEMO MODE
    types/index.ts # TypeScript types
    App.tsx
    dist/ # Built frontend
    backend/ # Flask Backend (Optional)
    app.py # Main Flask API
    requirements.txt
    README.md
```

Running the Project

Option 1: Demo Mode (No Backend Required!)

The deployed website works entirely in the browser with demo mode enabled. All features work without any backend!

Option 2: With Backend

Frontend Setup

```
cd app
npm install
npm run dev
```

Frontend at <http://localhost:5173>

Backend Setup (Optional)

```
cd backend
pip install -r requirements.txt
python app.py
```

Backend at <http://localhost:5000>

How to Use

1. Report an Emergency

1. Click “Report Emergency” button
2. Describe the situation (e.g., “Car accident, someone bleeding”)
3. Watch live AI analysis as you type!
4. Add location, photos, videos (optional)
5. Submit report

2. View AI Analysis

After submission, see:

- Which services are needed (Police/Ambulance/Fire)
- Severity level

- Confidence score
- Generated emergency message

3. Track Status

- Watch real-time status updates
- See progress through all 6 stages
- View estimated response time

4. View Records

- Click “View Records” to see all reports
- Select any record for detailed view
- See AI classification for each report

5. Responder Dashboard

- Click “Responder Dashboard”
- View all emergencies
- Update status of any report
- See statistics and analytics

6. Chatbot Help

- Click chat icon (bottom right)
- Ask for first aid help
- Get emergency numbers
- Navigate the system

API Endpoints (Backend)

Endpoint	Method	Description
/api/health	GET	Check API status
/api/emergency/submit	POST	Submit emergency report
/api/emergency/<id>	GET	Get emergency details
/api/emergency/<id>/status	GET	Get emergency status
/api/emergency/<id>/status	PUT	Update emergency status
/api/emergencies	GET	List all emergencies
/api/analyze	POST	Analyze text (no record)
/api/chatbot/message	POST	Chatbot message
/api/stats	GET	Dashboard statistics

Demo Mode Features

The demo mode in `api.ts` includes:

- Complete emergency classification logic
- In-memory storage for emergencies
- Simulated chatbot responses
- Status tracking and updates
- Statistics generation

To disable demo mode and use real backend:

```
// In src/services/api.ts
const DEMO_MODE = false; // Change to false
```

Screenshots

Home Page

- Animated hero with floating emergency icons
- Emergency numbers display
- Feature highlights

Report Form

- Live AI analysis as you type
- Voice input button
- Location capture
- File upload support

AI Analysis Results

- Service classification badges
- Severity indicator
- Confidence score
- Generated message

Status Tracker

- Visual timeline
- Progress bar
- Live updates
- Status history

Records Viewer

- List of all emergencies
- Filter by status/severity
- Detailed view
- Statistics summary

Responder Dashboard

- Real-time stats
- Emergency cards
- Status update controls
- Service breakdown

Browser Support

- Chrome 90+
- Firefox 88+
- Safari 14+
- Edge 90+

Team

Built with ❤️ for Vibe-a-Thon 2026

License

MIT License

Note: This is a hackathon project demonstration. For actual emergency situations, always call your local emergency numbers directly.

Emergency Numbers:

- 🚓 Police: **100**
- 🚑 Ambulance: **108**
- 🚒 Fire: **101**
- 🌐 Universal: **112**